M4GB end to end: the whole index-calculus pipeline on a curve ladder (2026-11-10)

M4GB end to end: the whole index-calculus pipeline on a curve ladder (2026-11-10)

Owner's ask after the descent ladder: a full end-to-end benchmark. Instrument: `ic bench --sweep` (src/bin/ic/bench.rs, `docs/ic/FRAMEWORK.md`): on one curve with planted logarithms it builds the factor base, runs the decomposition oracle until the relation matrix has rank, eliminates, recovers the logarithm and checks it against the planted one, and prices every phase in group-addition equivalents, `S = total / √r`, beside the matched counted Pollard rho (`--rho-runs 16`) on the same subgroup and targets (`s_over_rho`).

Status words. Local development check on the session container (4 vCPU Intel Xeon 2.10 GHz, 16 GiB), `[pcc:unverified]`; not isolab, not a leaderboard row (AGENTS.md validation gate). Benchmark record `benchmarks/records/20261110T1937Z-m4gb-e2e-ic-ladder.json`.

Cells

One sweep file per instance under `sweeps/` (each `ic bench --sweep` writes its own JSON under `results/`): Koblitz curves of degree 13, 17, 19, 23 and random binary curves of degree 15, 21 (`max_cofactor` 8). On each, the same `binary-subspace:dimension=⌈n/2⌉` factor base (`2⌈n/2⌉` boolean unknowns, `n` quadratics at `m = 2`) in front of:

- `mitm:m=2`, the pair-table oracle: the non-algebraic reference row;
- `descent-algebraic:m=2` with `inherited-f4` (the Koblitz oracle's degree-bounded hybrid), `f4-f2` (in-house full reduced basis) and `m4gb-f2` (M4GB upstream `01860a52` + patches 0001/0002, one thread, narrowest binary that fits, 60 s per call).

Repeats 2, seed 20261110, `--targets walk`, `incremental-gauss`; one repeat on the degree-23 cell: the degree-21 cell needed about 820 targets per repeat, so degree 23 needs about 1,700, and at about 2 s per `f4-f2` call two repeats would not fit the session's two-hour cap on one invocation.

How the solver's work enters S, and why S is not the engine comparison

`ic_framework` prices word-XOR counts at the repository's pinned `ns_per_word_xor / ns_per_add` ratio (`priced_by: pinned` for `inherited-f4` and `f4-f2` on these instances; the count covers their eliminations only, a third or less of their wall time at small sizes, as `solvers.rs` documents) and every other unit, M4GB's included (it reports no count), at measured wall time over the host's addition time (`priced_by: measured`). The smoke cell made the consequence plain: at 14 unknowns `f4-f2` takes 2.9 ms per call and `m4gb-f2` 16 ms, a 5.5× wall gap, while their S differ by 300× (424 against 128,100). So between a pinned-count engine and a wall-time engine **S compares pricing bases, not engines**. The engine comparison in this study is the solver's wall per call (both measured on this host); S is reported for every row because it is what the pipeline charges and what the rho column divides, and the `m4gb-f2` S rows are an upper bound in that sense.

Pre-stated expectation

From the per-call ladder (`research/m4gb_descent_ladder_20261110/`): the pair table and `inherited-f4` give the lowest S everywhere; `m4gb-f2` lands between `inherited-f4` and `f4-f2` from about 22 unknowns (degree 21 and up) and above both below that; every row recovers the planted logarithm (`verified: true`) or says why not `[pcc:unverified]`; every index-calculus row stays above the matched rho (`s_over_rho > 1`), as on every class measured so far. Falsifier: an `m4gb-f2` row with `S` below the `inherited-f4` row of the same cell, or any row below its matched rho.

Results

`RESULTS.md` (full table), `results.svg` / `results.png` (graph), `results.json`, `REPORT.pdf`; per-instance JSON under `results/`. Regenerate with `./make_report.sh`. Six instances, 24 end-to-end configurations (two repeats each except degree 23), **every one recovered its planted logarithm** and the matched rho verified on all 16 runs per instance. All numbers `[pcc:unverified]`, one thread, local container.

degree (family, unknowns, r)	rho S	pair table S	inherited-f4 S	f4-f2 S	m4gb-f2 S	ms per call: inherited / f4-f2 / m4gb	targets
13 (koblitz, 14 unknowns, r = 2003)	2.76	198	123	455	136908	1.0 / 2.9 / 16.1	42
15 (char2, 16 unknowns, r = 5503)	2.64	474	287	1736	124118	1.9 / 7.2 / 25.8	48
17 (koblitz, 18	2.02	493	249	1844	176669	3.1 / 25.4 / 46.7	122

degree (family, unknowns, r)	rho S	pair table S	inherited-f4 S	f4-f2 S	m4gb-f2 S	ms per call: inherited / f4-f2 / m4gb	targets
unknowns, r = 65587)							
19 (koblitz, 20 unknowns, r = 262543)	1.48	1063	411	6709	253620	5.3 / 65.1 / 74.0	244
21 (char2, 22 unknowns, r = 1049239)	1.36	1976	2912	705787	1285977	10.0 / 294.8 / 237.4	822
23 (koblitz, 24 unknowns, r = 4196903)	0.88	3986	2207	1657918	1834209	18.1 / 1733.4 / 462.4	1481

S = mean over repeats; ms per call = the solver's wall per decomposition call including, for M4GB, process start and the file round trip; targets = targets tried per repeat (hit rate 0.63 to 0.69 on every cell).

- **Every index-calculus row is far above the matched rho** (`S / rho [pcc:unverified]` from 45 at degree 13 to 2,500 for the best row at degree 23, and higher for the others), as on every class the framework has measured; nothing here changes that.
- **Best configuration per cell:** the algebraic oracle with `inherited-f4` on the Koblitz curves (S below the pair table at degrees 13, 17, 19, 23: 123 vs 198, 249 vs 493, 411 vs 1,063, 2,207 vs 3,986, with its word-XOR count priced at the pinned ratio); the pair table on the two random curves, where `inherited-f4` is priced by measured wall and lands 0.6× to 1.5× the pair table. `m4gb-f2` is never the best row.
- **M4GB against the in-house full-basis F4, per call (same pricing basis, wall):** 16 / 2.9 ms at 14 unknowns, 26 / 7.2 at 16, 47 / 25 at 18, 74 / 65 at 20, 237 / 295 at 22, 462 / 1,733 at 24. The crossover sits at 20 to 22 unknowns, as in the per-call ladder, and at 24 unknowns M4GB is 3.7× faster per call. Against `inherited-f4` it is 15 to 25× slower on every cell.
- **The S column says something else, and that is the pricing basis, not the engine.** At degree 23 `f4-f2` is 3.7× slower per call than M4GB yet its S (1.66 M, pinned eliminations only) is below M4GB's (1.83 M, whole wall). The pinned count covers a fraction of `f4-f2`'s run; M4GB's row charges everything. A like-for-like S would need a counted unit inside M4GB (the open thread from `research/m4gb_integration_20261110/`).
- **Pre-stated expectation:** held. `inherited-f4` and the pair table give the lowest S everywhere, `m4gb-f2` lands between `inherited-f4` and `f4-f2` on wall per call from about 22 unknowns and above both below that, every row verified, every row above rho. The falsifier (an `m4gb-f2` row with S below the `inherited-f4` row, or any row below rho) did not fire.

- **Where the time goes end to end:** the solver is 88 to 100 % of each algebraic row's GAE; with the subspace base at dimension `[n/2]` the hit rate is about 0.65, so the pipeline needs only about 1.6 targets per relation and the whole run is the decomposition cost. Per-call solver speed is therefore the whole story for this oracle, which is why the per-call ladder and this end-to-end ladder agree.

Open threads

- Isolab re-run on a pinned image (this container shared CPU with nothing else during the run, but it is not the lab).
- A persistent M4GB process (one spawn, many systems over a pipe) would remove the 8 to 16 ms start-up that dominates its small-cell cost; and a counted unit inside M4GB would let S compare engines rather than pricing bases.
- The Gröbner-trace idea for this oracle: the 1,481 systems of one cell differ only in the target's constants, so the sequence of critical pairs and reducers that one solve discovers could be replayed on the next (Traverso's trace); `inherited-f4` already exploits the shared structure and is the engine to compare against.

Per instance: the matched counted rho (mean S over its runs) and every configuration run end to end. `S` = total group-addition equivalents / sqrt(r), mean over repeats (min–max); `S / rho` = mean s_over_rho; `ok` = every repeat recovered the planted logarithm; `hit` = decomposition hit rate; `tried` = targets tried; `solver` = engine, priced_by, mean calls, budget hits, share of the row's GAE, wall per call.

instance (r)	configuration	S mean (min–max)	S
`icv1-f2m15-tm249-9e24ae34` (r = 5503)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	2.64	
	`binary-subspace[dimension=8] + mitm[m=2] + – + walk`	473.84 (473.10–474.59)	
	`binary-subspace[dimension=8] + descent-algebraic[m=2] + inherited-f4 + walk`	287.31 (195.56–379.06)	
	`binary-subspace[dimension=8] + descent-algebraic[m=2] + f4-f2 + walk`	1735.90 (1166.62–2305.18)	

instance (r)	configuration	S mean (min–max)	S
	`binary-subspace[dimension=8] + descent-algebraic[m=2] + m4gb-f2 + walk`	124118.48 (84542.80–163694.16)	47(
`icv1-f2m21-tm1325-b6925563` (r = 1049239)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	1.36	
	`binary-subspace[dimension=11] + mitm[m=2] + – + walk`	1975.73 (1975.73–1975.74)	1(
	`binary-subspace[dimension=11] + descent-algebraic[m=2] + inherited-f4 + walk`	2911.76 (2874.91–2948.62)	2(
	`binary-subspace[dimension=11] + descent-algebraic[m=2] + f4-f2 + walk`	705787.23 (705782.84–705791.63)	517(
	`binary-subspace[dimension=11] + descent-algebraic[m=2] + m4gb-f2 + walk`	1285977.24 (1254182.51–1317771.98)	943(
`icv1-f2m13-t181-515ee569` (r = 2003)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	2.76	
	`binary-subspace[dimension=7] + mitm[m=2] + – + walk`	197.66 (197.66–197.66)	

instance (r)	configuration	S mean (min–max)	S
	`binary-subspace[dimension=7] + descent-algebraic[m=2] + inherited-f4 + walk`	123.11 (117.79–128.43)	
	`binary-subspace[dimension=7] + descent-algebraic[m=2] + f4-f2 + walk`	455.44 (424.38–486.50)	1
	`binary-subspace[dimension=7] + descent-algebraic[m=2] + m4gb-f2 + walk`	136908.29 (128520.02–145296.57)	49
`icv1-f2m17-tm101-00378d4e` (r = 65587)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	2.02	
	`binary-subspace[dimension=9] + mitm[m=2] + – + walk`	493.04 (492.55–493.54)	2
	`binary-subspace[dimension=9] + descent-algebraic[m=2] + inherited-f4 + walk`	249.09 (102.55–395.63)	1
	`binary-subspace[dimension=9] + descent-algebraic[m=2] + f4-f2 + walk`	1843.65 (745.30–2942.00)	9
	`binary-subspace[dimension=9] + descent-algebraic[m=2] + m4gb-f2 + walk`	176669.07 (72915.26–280422.88)	87

instance (r)	configuration	S mean (min–max)	S
icv1-f2m19-tm797-9c54981b (r = 262543)	**matched rho** `negation-map r-adding walk: look-ahead, short-cycle escape by doubling, distinguished points, stride starts`, 16 runs	1.48	
	`binary-subspace[dimension=10] + mitm[m=2] + – + walk`	1063.15 (1062.78–1063.52)	1
	`binary-subspace[dimension=10] + descent-algebraic[m=2] + inherited-f4 + walk`	411.01 (227.52–594.50)	2
	`binary-subspace[dimension=10] + descent-algebraic[m=2] + f4-f2 + walk`	6709.06 (3567.97–9850.15)	45
	`binary-subspace[dimension=10] + descent-algebraic[m=2] + m4gb-f2 + walk`	253620.31 (135112.69–372127.93)	171
icv1-f2m23-tm5197-1f85e9e1 (r = 4196903)	**matched rho** `signed-Frobenius r-adding walk, distinguished points (koblitz_signed_frobenius_rho_reference)`, 16 runs	0.88	
	`binary-subspace[dimension=12] + mitm[m=2] + – + walk`	3986.40 (3986.40–3986.40)	45
	`binary-subspace[dimension=12] + descent-algebraic[m=2] + inherited-f4 + walk`	2206.73 (2206.73–2206.73)	25

instance (r)	configuration	S mean (min–max)	S
	`binary-subspace[dimension=12] + descent-algebraic[m=2] + f4-f2 + walk`	1657917.78 (1657917.78–1657917.78)	1894
	`binary-subspace[dimension=12] + descent-algebraic[m=2] + m4gb-f2 + walk`	1834208.82 (1834208.82–1834208.82)	2095